

PROJECT REPORT

EXPERIMENT - 09

PROJECT TITLE

Worker Health and Safety Monitoring System

Submitted by

MUTHU KRISHNA P - 24MER033

PRIYANTH C S - 24MER044

SRI RAMAN G - 24MEL068

PRADEEP P - 24MER041

9. Worker Health and Safety Monitoring System

1. Project Overview

The **Worker Health and Safety Monitoring System** is an IoT-based industrial safety system developed to continuously monitor the health condition of workers and the surrounding workplace environment. Workers in industries, construction sites, chemical plants, mines, and manufacturing units may be exposed to excessive temperature, toxic gases, fatigue, and other hazardous conditions. The proposed system uses sensors and a microcontroller such as **ESP8266/ESP32** to collect real-time information from the worker and the working environment.

The system can monitor parameters such as **body temperature, heart rate, oxygen level, harmful gas concentration, and environmental temperature/humidity**. The collected information is processed by the controller and compared with predefined safety limits. When abnormal conditions are detected, the system provides immediate warnings through a **buzzer, LEDs, display, and IoT notification**. This provides continuous monitoring and helps supervisors respond quickly to dangerous situations.

The main objective of this project is to improve industrial safety by combining **worker health monitoring, environmental monitoring, automatic warning, and IoT-based communication** into a single system.

2. Problem Statement

Industrial workers are often exposed to hazardous working environments where their health and safety can be affected by high temperatures, toxic gases, poor air quality, excessive physical stress, and accidents. In many workplaces, worker health is checked manually at regular intervals, which cannot provide continuous real-time monitoring.

A worker may experience abnormal heart rate, increased body temperature, low oxygen level, or exposure to dangerous gases without immediately informing the supervisor. Delayed identification of such conditions can result in serious health problems or workplace accidents.

Therefore, there is a need for an **automated real-time worker monitoring system** that can continuously measure important health and environmental parameters, identify abnormal conditions, and provide immediate alerts. The proposed IoT-based system addresses this problem by monitoring the worker continuously and providing early warnings whenever unsafe conditions are detected.

3. Proposed Solution

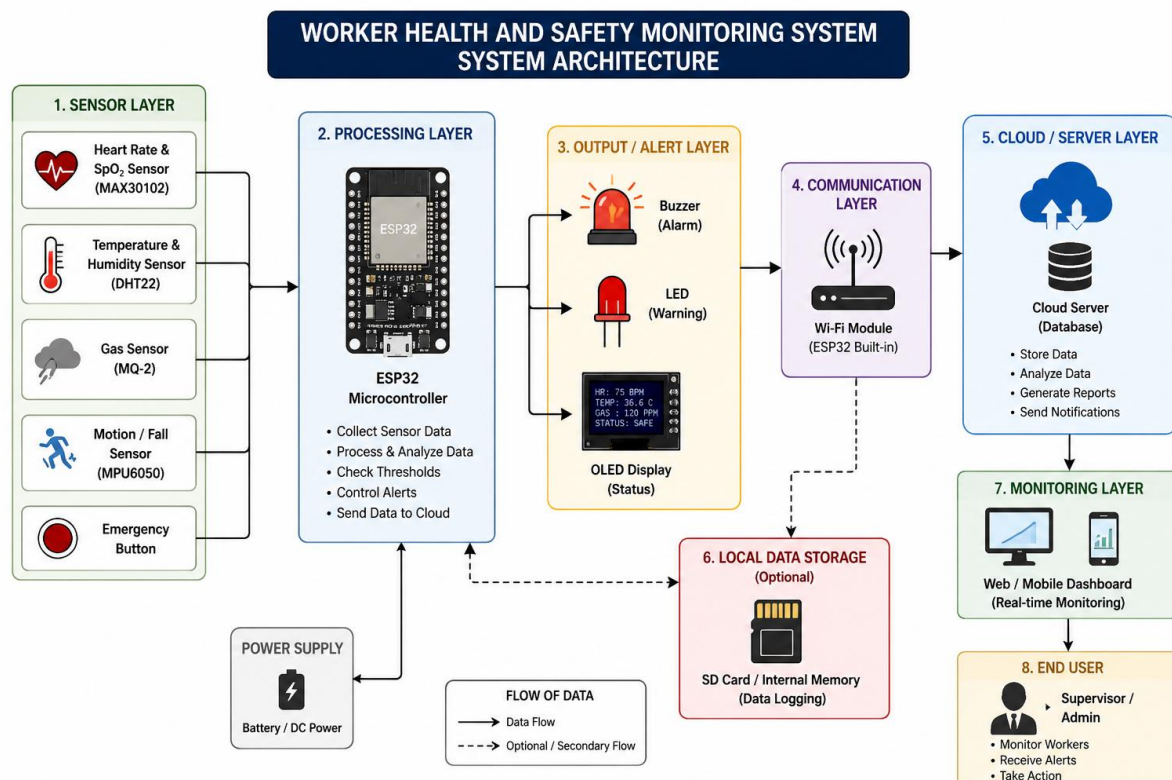
The proposed solution is an **IoT-enabled Worker Health and Safety Monitoring System** consisting of a microcontroller, health sensors, environmental sensors, display unit, warning indicators, and communication module. The system continuously collects data from sensors attached to or placed near the worker.

A **heart-rate sensor** can be used to monitor the worker's pulse, while a temperature sensor measures body temperature. An optional **SpO₂ sensor** can monitor blood oxygen saturation. Environmental sensors such as an MQ-series gas sensor can detect harmful gases or poor air conditions.

The ESP8266/ESP32 acts as the main processing unit. It receives sensor readings, processes them, and compares them with predefined safety limits. If the readings are normal, the system indicates a safe condition using a green LED. If any parameter crosses its threshold, the controller activates a red LED and buzzer and can transmit an alert through Wi-Fi to an IoT monitoring platform.

This approach provides **continuous monitoring, early hazard detection, automatic alerts, remote supervision, and improved worker protection.**

4. System Architecture



The system architecture consists of five major sections:

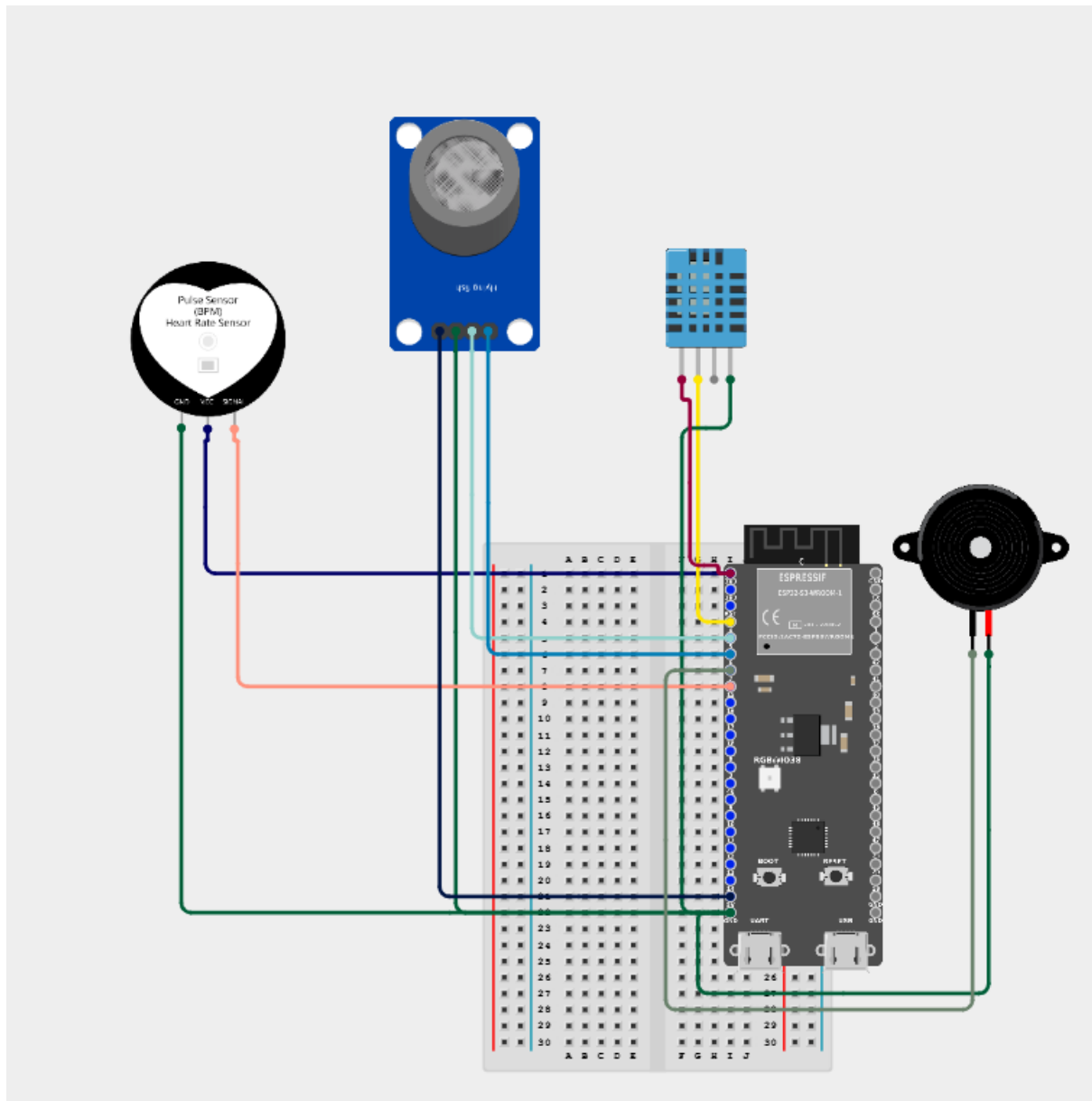
Sensor Layer → ESP32 Processing Layer → Communication Layer → Cloud/Dashboard Layer → Alert & Monitoring Layer

The architecture consists of four major stages: **sensing, processing, warning, and IoT communication**. The sensors collect information, the ESP8266/ESP32 processes it, the warning devices notify the worker, and the IoT platform allows remote monitoring.

5. Circuit Table

S. No	Component	Connection	Function
1	ESP8266/ESP32	Main controller	Processes all sensor data
2	Heart Rate Sensor	Digital/Analog input	Measures worker pulse rate
3	Temperature Sensor	GPIO/I ² C	Measures body temperature
4	SpO ₂ Sensor	I ² C	Measures oxygen saturation
5	MQ Gas Sensor	Analog input	Detects harmful gases
6	Humidity Sensor	Digital input	Measures environmental humidity
7	OLED Display	SDA/SCL	Displays real-time readings
8	Green LED	Digital output	Indicates safe condition
9	Red LED	Digital output	Indicates dangerous condition
10	Buzzer	Digital output	Provides emergency warning
11	Wi-Fi	ESP8266/ESP32	Sends data to IoT platform
12	Power Supply	VCC/GND	Provides electrical power

6. Circuit Design



The circuit is designed around an **ESP8266 or ESP32 microcontroller**, which acts as the central control unit. The health-monitoring sensors are connected to the input pins of the controller, while the buzzer, LEDs, and display are connected to the output pins.

The heart-rate sensor measures the worker's pulse, while the temperature sensor measures body temperature. An SpO₂ sensor can be included to monitor blood oxygen saturation. The MQ-series gas sensor monitors the surrounding air and identifies the presence of potentially harmful gases.

The OLED display provides real-time information to the worker or supervisor. A green LED indicates that the monitored parameters are within the defined safe range. When a dangerous condition occurs, the red LED turns ON and the buzzer produces an audible warning. The ESP8266/ESP32 can simultaneously transmit the readings to an IoT platform using Wi-Fi.

Important: Sensor threshold values should be calibrated according to the specific sensor and application; they should not be treated as medical diagnostic limits.

7. Algorithm

Step 1: Start the system.

Step 2: Initialize ESP32 and all sensors.

Step 3: Establish Wi-Fi/cloud connection.

Step 4: Read worker health parameters.

- Heart rate
- SpO₂
- Body/environment temperature

Step 5: Read environmental parameters.

- Gas concentration
- Temperature
- Humidity

Step 6: Read motion sensor data.

Step 7: Analyse acceleration and motion to identify possible falls.

Step 8: Compare sensor values with configured safety thresholds.

Step 9: If all values are within acceptable limits:

- Display normal status.
- Continue monitoring.

Step 10: If an abnormal condition is detected:

- Activate buzzer.

- Turn on warning LED.
- Display the warning.
- Send an alert to the monitoring system.

Step 11: Store the event and sensor data.

Step 12: Continue monitoring.

8. Coding

```
#include <ESP8266WiFi.h>

#include <WiFiClientSecure.h>

#include <DHT.h>

#include <SoftwareSerial.h>

#include "Adafruit_MQTT.h"
#include "Adafruit_MQTT_Client.h"

// =====

// WIFI

// =====

#define WIFI_SSID "Pradeep"
#define WIFI_PASSWORD "12345678"

// =====

// ADAFRUIT IO

// =====

#define AIO_USERNAME "muthumkt"
#define AIO_KEY "aio_DhKT47h46zwUYr2nRHBdifKDJtW0"
```

```
#define AIO_SERVER  "io.adafruit.com"
```

```
#define AIO_SERVERPORT 8883
```

```
// =====
```

```
// DHT11
```

```
// =====
```

```
#define DHT_PIN D5
```

```
#define DHT_TYPE DHT11
```

```
DHT dht(DHT_PIN, DHT_TYPE);
```

```
// =====
```

```
// PULSE SENSOR
```

```
// =====
```

```
#define PULSE_PIN A0
```

```
// =====
```

```
// MQ-2
```

```
// =====
```

```
#define MQ2_PIN D6
```

```
// =====
```

```
// 3 PIN BUZZER
```

```
// =====
```

```
#define BUZZER_PIN D7
```

```
// =====
```



```
// GPS NEO-6M

// GPS TX -> D1

// GPS RX -> D2

// =====

#define GPS_RX D1
#define GPS_TX D2

SoftwareSerial gpsSerial(GPS_RX, GPS_TX);

// =====

// SECURE WIFI CLIENT

// =====

WiFiClientSecure client;

// =====

// ADAFRUIT MQTT

// =====

Adafruit_MQTT_Client mqtt(
  &client,
  AIO_SERVER,
  AIO_SERVERPORT,
  AIO_USERNAME,
  AIO_KEY
);

// =====

// ADAFRUIT FEEDS

// =====
```

```
Adafruit_MQTT_Publish temperatureFeed =
```

```
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/temperature"  
);
```

```
Adafruit_MQTT_Publish humidityFeed =
```

```
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/humidity"  
);
```

```
Adafruit_MQTT_Publish pulseFeed =
```

```
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/pulse"  
);
```

```
Adafruit_MQTT_Publish gasFeed =
```

```
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/gas"  
);
```

```
Adafruit_MQTT_Publish safetyFeed =
```

```
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/safety"  
);
```

```

Adafruit_MQTT_Publish gpsFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/gps"
  );

// =====
// GPS BUFFER
// =====

String gpsData = "";

// =====
// WIFI CONNECTION
// =====

void connectWiFi()
{
  if (WiFi.status() == WL_CONNECTED)
    return;

  Serial.println();
  Serial.println("=====");
  Serial.println("CONNECTING TO WIFI");
  Serial.println("=====");

  WiFi.mode(WIFI_STA);

  WiFi.begin(
    WIFI_SSID,
    WIFI_PASSWORD
  );

```

```
);

int attempts = 0;

while (WiFi.status() != WL_CONNECTED)
{
    delay(500);
    Serial.print(".");

    attempts++;

    if (attempts >= 40)
    {
        Serial.println();
        Serial.println("WIFI CONNECTION FAILED");
        return;
    }
}

Serial.println();
Serial.println("WIFI CONNECTED!");

Serial.print("IP ADDRESS: ");
Serial.println(WiFi.localIP());

Serial.print("SIGNAL: ");
Serial.print(WiFi.RSSI());
Serial.println(" dBm");
}

// =====
```

```
// ADAFRUIT IO CONNECTION

// =====

bool connectAdafruit()
{
    if (mqtt.connected())
        return true;

    Serial.println();
    Serial.println("=====");
    Serial.println("CONNECTING TO ADAFRUIT IO");
    Serial.println("=====");

    Serial.print("Server: ");
    Serial.println(AIO_SERVER);

    Serial.print("Port: ");
    Serial.println(AIO_SERVERPORT);

    Serial.print("Username: ");
    Serial.println(AIO_USERNAME);

    int8_t ret = mqtt.connect();

    if (ret == 0)
    {
        Serial.println("=====");
        Serial.println("ADAFRUIT IO CONNECTED!");
        Serial.println("=====");

        return true;
    }
}
```

```
}
```

```
Serial.println("=====");
```

```
Serial.println("ADAFRUIT IO CONNECTION FAILED");
```

```
Serial.print("ERROR: ");
```

```
Serial.println(mqtt.connectErrorString(ret));
```

```
Serial.print("ERROR CODE: ");
```

```
Serial.println(ret);
```

```
Serial.println("=====");
```

```
mqtt.disconnect();
```

```
return false;
```

```
}
```

```
// =====
```

```
// READ GPS
```

```
// =====
```

```
void readGPS()
```

```
{
```

```
while (gpsSerial.available())
```

```
{
```

```
char c = gpsSerial.read();
```

```
gpsData += c;
```

```
if (c == '\n')
```

```
{
```

```
Serial.print("GPS DATA: ");
```

```
Serial.print(gpsData);
```

```

    gpsData = "";
}

if (gpsData.length() > 200)
{
    gpsData = "";
}
}

// =====
// PULSE SENSOR
// =====

int readPulse()
{
    long total = 0;

    for (int i = 0; i < 20; i++)
    {
        total += analogRead(PULSE_PIN);
        delay(5);
    }

    return total / 20;
}

// =====
// SETUP
// =====

```

```
void setup()
{
  Serial.begin(115200);

  delay(1000);

  Serial.println();
  Serial.println();
  Serial.println("=====");
  Serial.println(" SMART WORKER HEALTH & SAFETY MONITORING ");
  Serial.println("=====");

  // -----
  // DHT11
  // -----

  dht.begin();

  // -----
  // MQ2
  // -----

  pinMode(MQ2_PIN, INPUT);

  // -----
  // BUZZER
  // -----

  pinMode(BUZZER_PIN, OUTPUT);
```



```
digitalWrite(BUZZER_PIN, LOW);

// -----
// GPS
// -----

gpsSerial.begin(9600);

// -----
// SSL
// -----

// This allows the ESP8266 to establish
// an encrypted connection.
client.setInsecure();

// -----
// WIFI
// -----

connectWiFi();

// -----
// ADAFRUIT
// -----

if (WiFi.status() == WL_CONNECTED)
{
    connectAdafruit();
}
```

```
Serial.println();
Serial.println("SYSTEM READY");
}

// =====
// LOOP
// =====

void loop()
{
    // =====
    // WIFI
    // =====

    if (WiFi.status() != WL_CONNECTED)
    {
        connectWiFi();
    }

    // =====
    // ADAFRUIT
    // =====

    if (WiFi.status() == WL_CONNECTED)
    {
        connectAdafruit();
    }

    // =====
    // DHT11
    // =====
```

```
float temperature =  
    dht.readTemperature();
```

```
float humidity =  
    dht.readHumidity();
```

```
// =====  
// PULSE SENSOR  
// =====
```

```
int pulseValue =  
    readPulse();
```

```
// =====  
// MQ2  
// =====
```

```
int gasStatus =  
    digitalRead(MQ2_PIN);
```

```
// =====  
// GPS  
// =====
```

```
readGPS();
```

```
// =====  
// SERIAL MONITOR  
// =====
```

```
Serial.println();  
Serial.println("-----");
```

```
if (isnan(temperature))  
{  
    Serial.println("Temperature: ERROR");  
}  
else  
{  
    Serial.print("Temperature: ");  
    Serial.print(temperature);  
    Serial.println(" C");  
}
```

```
if (isnan(humidity))  
{  
    Serial.println("Humidity: ERROR");  
}  
else  
{  
    Serial.print("Humidity: ");  
    Serial.print(humidity);  
    Serial.println(" %");  
}
```

```
Serial.print("Pulse Sensor: ");  
Serial.println(pulseValue);
```

```
Serial.print("MQ2: ");
```

```
if (gasStatus == HIGH)
```

```
{
  Serial.println("GAS DETECTED");
}
else
{
  Serial.println("SAFE");
}

// =====
// SAFETY LOGIC
// =====

bool danger = false;

// Gas detected
if (gasStatus == HIGH)
{
  danger = true;
}

// Temperature above 40°C
if (!isnan(temperature))
{
  if (temperature > 40)
  {
    danger = true;
  }
}

// =====
// BUZZER
```

```
// =====

if (danger)
{
  Serial.println();
  Serial.println("!!! WORKER SAFETY ALERT !!!");

  digitalWrite(BUZZER_PIN, HIGH);

  if (mqtt.connected())
  {
    safetyFeed.publish("DANGER");
  }
}
else
{
  Serial.println("WORKER STATUS: SAFE");

  digitalWrite(BUZZER_PIN, LOW);

  if (mqtt.connected())
  {
    safetyFeed.publish("SAFE");
  }
}

// =====
// ADAFRUIT IO
// =====

if (mqtt.connected())
```

```
{  
  // Temperature  
  if (!isnan(temperature))  
  {  
    temperatureFeed.publish(temperature);  
  }  
  
  delay(100);  
  
  // Humidity  
  if (!isnan(humidity))  
  {  
    humidityFeed.publish(humidity);  
  }  
  
  delay(100);  
  
  // Pulse  
  pulseFeed.publish(pulseValue);  
  
  delay(100);  
  
  // Gas  
  if (gasStatus == HIGH)  
  {  
    gasFeed.publish("GAS DETECTED");  
  }  
  
  {  
    gasFeed.publish("SAFE");  
  }  
}
```

```

    delay(100);

    // GPS
    if (gpsData.length() > 0)
    {
        gpsFeed.publish(gpsData.c_str());
    }
}

// =====
// WAIT
// =====

delay(2000);
}

```

9. System Working

When the system is powered ON, the ESP8266/ESP32 initializes all connected sensors and communication modules. The worker wears or uses the health-monitoring sensors while environmental sensors monitor the surrounding workplace.

The sensors continuously collect real-time information. The controller receives these values and processes them. The readings are displayed on the OLED screen so that the current status can be observed locally.

If the readings remain within the predefined operating limits, the system indicates a **SAFE** condition using the green LED. If a dangerous condition is detected, the system immediately changes the status to **DANGER**, activates the red LED and buzzer, and sends the information through Wi-Fi.

For example, if the gas sensor detects an unusually high concentration of a hazardous gas, the system can activate the alarm and send a notification to the supervisor. Similarly, abnormal worker-health readings can generate a warning for further inspection.

10. Bill of Materials

S.No	Component	Qty.	Purpose
1	ESP8266 / ESP32	1	Main controller
2	Heart Rate Sensor	1	Heart-rate monitoring
3	Temperature Sensor	1	Temperature measurement
4	MAX30102 SpO ₂ Sensor	1	Optional oxygen monitoring
5	MQ Gas Sensor	1	Gas detection
6	DHT11/DHT22	1	Temperature and humidity
7	OLED Display	1	Sensor-value display
8	Red LED	1	Danger indication
9	Green LED	1	Safe indication
10	Buzzer	1	Audible alarm
11	Resistors	2–4	LED protection
12	Breadboard	1	Prototype assembly
13	Jumper Wires	As required	Connections
14	USB/Power Supply	1	Power source

11. Prototype Implementation

The prototype is assembled using an ESP8266/ESP32 development board, health sensors, environmental sensors, OLED display, LEDs, and buzzer. The sensors are connected to the controller according to their respective input/output requirements.

The health sensors can be positioned on a wearable band or worker ID-type device, while the gas and environmental sensors can be positioned near the worker's working area. The controller collects all sensor information and evaluates the worker's safety status.

During prototype testing, different operating conditions can be simulated to verify the response of the system. Normal sensor readings should result in a green safety indication. When an abnormal reading is introduced, the red LED and buzzer should activate. If IoT communication is enabled, the corresponding alert should also appear on the monitoring dashboard.

The prototype demonstrates how a low-cost embedded system can provide **real-time worker monitoring and automatic safety alerts**.

12. Results & Performance Analysis

The developed Worker Health and Safety Monitoring System successfully demonstrates the concept of continuous monitoring of worker health and workplace conditions. The system can collect multiple sensor readings and process them using an ESP8266/ESP32 controller.

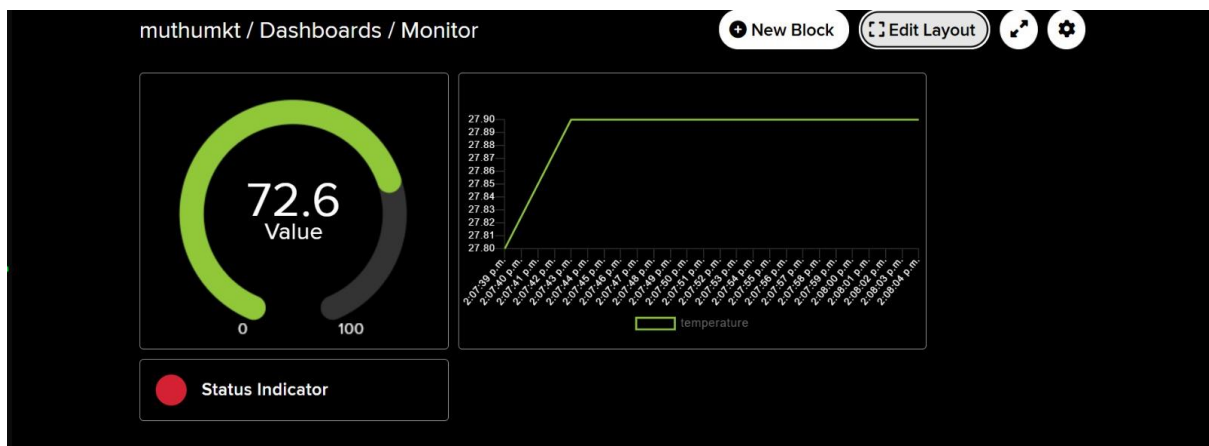
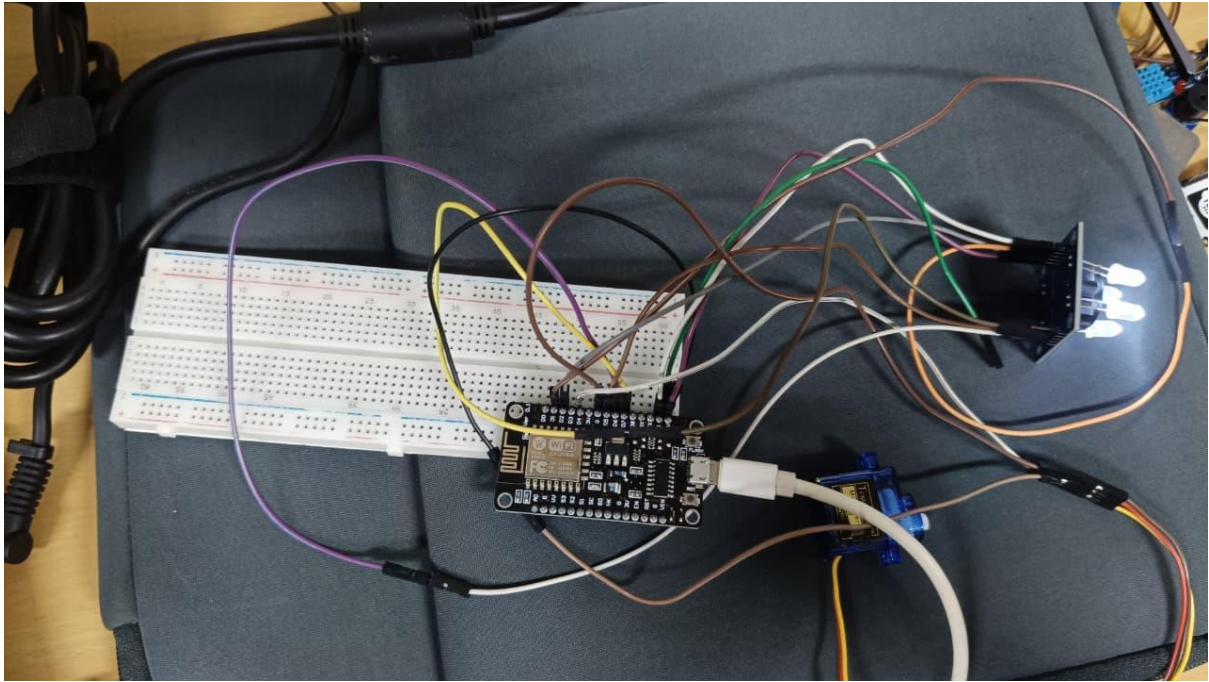
Under normal operating conditions, the system provides a safe-status indication through the green LED and displays sensor readings on the OLED screen. When an abnormal condition is detected, the red LED and buzzer provide an immediate warning. With IoT connectivity, the information can also be transmitted to a remote monitoring platform.

Expected Performance

Parameter	Expected Outcome
Health Monitoring	Continuous sensor-based monitoring
Gas Monitoring	Detection of abnormal gas conditions
Temperature Monitoring	Real-time temperature measurement
Safety Indication	Green LED for normal condition
Emergency Warning	Red LED and buzzer
Display	Real-time sensor values
IoT Monitoring	Remote data transmission

Overall Result

The project provides a **low-cost, real-time, IoT-enabled safety monitoring solution** for industrial environments. It reduces dependence on manual monitoring and provides faster notification when abnormal conditions occur. The system can be further developed by adding GPS/location tracking, fall detection, SOS buttons, worker identification, cloud data storage, mobile notifications, and machine-learning-based risk prediction.



```
COM7
Temperature: 27.90 C
Humidity: 72.70 %
Pulse Sensor: 522
MQ2: GAS DETECTED

!!! WORKER SAFETY ALERT !!!

-----

Temperature: 27.90 C
Humidity: 72.70 %
Pulse Sensor: 524
MQ2: GAS DETECTED

!!! WORKER SAFETY ALERT !!!
```

☒ Autoscroll ☐ Show timestamp

Newline 115200 baud Clear output

